Threadripper Build Notes

Last Modified Sun July 26 2026

Build Order

Core GPU/MPI Libraries

- Nvidia Driver
- CUDA Toolkit
- UCX
- libevent
- hwloc
- OpenPMIx
- PRRTE
- OpenMPI

OpenFOAM Tools

- ParaView
- Umpire
- Hypre
- Apt Installs (CGAL, Boost, etc)
- FFTW
- KaHIP
- GKlib
- METIS
- ParMETIS
- Scotch
- Zoltan
- AMGX
- PETSc
- OpenFOAM
- AmgX4Foam

`.bashrc` `PATH`, and `LD_LIBRARY_PATH` Variables

Inside `~/.bashrc`, don't forget to track changes to these variables following installation of each tool, e.g.:

```
...
export PATH=/opt/mytool/bin:$PATH
export LD_LIBRARY_PATH=/opt/mytool/lib:$LD_LIBRARY_PATH
```

Nvidia driver

Start with a fresh install of 24.04.4. TR uses Kernel 7 for driver 580-open

```
nvidia-smi
NVIDIA-SMI 580.173.02   Driver Version: 580.173.02  CUDA Version: 13.0
0  NVIDIA GeForce RTX 3090 Ti
1  NVIDIA GeForce RTX 5090

nvidia-smi --query-gpu=gpu_name,compute_cap --format=csv
name, compute_cap
NVIDIA GeForce RTX 3090 Ti, 8.6
```

```
NVIDIA GeForce RTX 5090, 12.0
```

Nvidia Prerequisites

```
# register the CUDA keyring
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2404/x86_64
    /cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update

# Threadripper needs Compute Architecture sm_86 and sm_120
sudo apt install nvidia-driver-580-open
```

CUDA Toolkit

```
# maintain Haswell's sm_61 compatibility, hence Toolkit 12.8
sudo apt install cuda-toolkit-12-8
reboot
```

`CUDA_HOME` Variable

Inside `~/.bashrc`, track changes as each tool is installed.

```
...
export CUDA_HOME=/usr/local/cuda
export PATH=$CUDA_HOME/bin
export LD_LIBRARY_PATH=$CUDA_HOME/lib64
```

UCX

Prerequisites

```
mkdir repos && cd repos

# collect the codebase
git clone --recursive https://github.com/openucx/ucx
cd ucx
sudo apt update
sudo apt install build-essential automake autoconf pkg-config libtool
```

Configure UCX

```
./autogen.sh

# For Threadripper:
./configure --prefix=/opt/ucx --with-cuda=/usr/local/cuda --with-nvcc-gencode
    ="-gencode=arch=compute_86,code=sm_86 -gencode=arch=compute_120,code=sm_120
    " --enable-mt
```

OpenMPI Dependencies

libevent

```
cd libevent
./autogen.sh
./configure --help
mkdir build && cd build
../configure --prefix=/opt/libevent
make
make verify
sudo make install
```

hwloc

```
cd hwloc
git checkout tags/hwloc-2.14.0
./autogen.sh
./configure --help
mkdir build && cd build
../configure --with-cuda=/usr/local/cuda --prefix=/opt/hwloc
make
sudo make install
```

Prerequisites

```
sudo apt install perl python3 python3-pip virtualenv flex bison gfortran
```

OpenPMIx

```
cd openpmix
git checkout tags/v6.1.1rc2
./autogen.pl
./configure --help
mkdir build && cd build
../configure --prefix=/opt/pmix --with-hwloc=/opt/hwloc --with-libevent=/opt/
    libevent
make -j$(nproc)
sudo make install
```

PRRTE

```
cd prrte
./autogen.pl
mkdir build && cd build
../configure --prefix=/opt/prrte --with-libevent=/opt/libevent --with-hwloc=/
    opt/hwloc --with-pmix=/opt/pmix
make -j$(nproc)
sudo make install
```

OpenMPI

```
cd ompi/
./autogen.pl
mkdir build && cd build
../configure --prefix=/opt/ompi --with-libevent=/opt/libevent --with-hwloc=/opt
    /hwloc --with-pmix=/opt/pmix --with-prrte=/opt/prrte --with-ucx=/opt/ucx --
    with-ucx-libdir=/opt/ucx/lib --with-cuda=/usr/local/cuda --with-cuda-libdir
    =/usr/local/cuda/lib64/stubs
make -j$(nproc)
sudo make install
```

Changes to `etc/openmpi-mca-params.conf`

This appears to be an important tweak for a custom UCX set-up with multiple gencodes. Doing this prevents `ob1` from being the highest prioritized provider in the stack. Instead, `ucx` is always picked. This helps in most cases and passes the PETSc tests, but in theory this configuration could also cause errors too. It is still a little early to say anything certain.

```
# Point-to-point Messaging Layer
pml=ucx
# One-sided Communication Layer
osc=ucx
```

OpenMPI `etc/prefs.sh` Changes

```
export MPI_ARCH_PATH=/opt/ompi
```

OpenFOAM `.com` External Tools

Introduction

Similar to most `make` tools, `Allwmake` looks for an `Allwmake` file and only builds subfolders containing one. You can run `./Allwmake` multiple times and it just picks up where it left off.

Hacking `etc/config.sh/*`

OpenFOAM has several quirks regarding its external dependencies.

OpenFOAM uses the convention `etc/config.sh/tool_name` to set default external tool folder locations, and a optional `prefs.sh` file inside `etc/` for preferences. So in theory, any configuration changes in `prefs.sh` get picked up during the standard calls to `./Allwmake`. You do not have to put `prefs.sh` there, and calling `foamEtcFile -list` shows all the searched locations.

However, when I tried calling `./Allwmake` when inside a subfolder, for some reason, it didn't find the `prefs.sh` file and reverted to OpenFOAM's default locations for those tools. But there was also some strange behavior involving the `prefs.sh` file being removed from `etc/` at some point, which may have caused the issue.

Eventually, I just hard-coded some of the paths inside `etc/config.sh/*` directly to push through and complete the build. I also had to add `export FFTW_DIR=/opt/fftw` to the `etc/config.sh/FFTW` folder.

Ignored Third-Party Tools

- ADIOS2
- CCMIO

- HDF5
- MGridGen

ParaView

Stale ParaView Libraries

I built the current version of ParaView after trying `v5.12.1`, the version used in the tarball associated with OpenFOAM `v2606` on OpenFOAM.com's website. Note that brittle dependencies exist between VTK and QT 5 in the only ParaView-aware OpenFOAM component, a module named PVFoamReader. PVFoamReader is also the only tool that requires HDF5 support. There is also an associated visualization component, an in-process off-screen renderer, that uses VTK libraries as well. It may have built on Haswell without issues.

Due to the issues with those incompatible ParaView libraries, two `Allwmake` files were renamed to `xxxAllwmake`, in `src/plugins/bindings/vtk-hdf` and `src/modules/visualization`.

Pre-Requisites

```
sudo apt install cmake ninja-build libtbb-dev mesa-common-dev mesa-utils
    freeglut3-dev xsltproc libxkbcommon-dev qt6-5compat-dev qt6-base-dev qt6-
    tools-dev qt6-svg-dev
```

Cmake Configuration

```
cd repos
git clone --recursive https://github.com/Kitware/ParaView.git
cd ParaView
mkdir paraview-build && cd paraview-build
cmake -GNinja -DCMAKE_INSTALL_PREFIX=/opt/paraview -DPARAVIEW_USE_PYTHON=ON -
    DPARAVIEW_USE_CUDA=ON -DPARAVIEW_USE_MPI=ON -DCMAKE_CUDA_ARCHITECTURES="
    native" -DVTK_SMP_IMPLEMENTATION_TYPE=TBB -DCMAKE_BUILD_TYPE=Release ../

# ninja -j runs out of memory and always has to be re-run a couple of times on
    both machines.
# build feedback is minimal. some libraries take several minutes to build.
# Update: ninja -j$(nproc) ultimately failed on TR.
# I had to use cmake --build . -j$(nproc) to finally complete the process.
# Apparently, there are known issues with how ninja handles large core sizes on
     some AMD CPUs.

# ninja -j$(nproc)
cmake --build . -j$(nproc)
sudo ninja install
```

ParaView and VTK `etc/prefs.sh` Changes

```
export ParaView_VERSION="none"
export VTK_VERSION="none"
```

Umpire

Like PRRTE, and OpenPMIx, Umpire is used by Hypre and is a configurable installation inside Hypre's build process. Here is the authoritative source.

Collect Umpire Code

```
git clone --recursive https://github.com/LLNL/Umpire.git
cd Umpire
mkdir build && cd build
```

Cmake Configuration and Build

```
cmake -DENABLE_MPI=ON -DENABLE_CUDA=ON -DBLT_ENABLE_CUDA=ON -DBLT_ENABLE_OPENMP
    =ON -DBLT_ENABLE_MPI=ON -DBUILD_SHARED_LIBS=ON -DUMPIRE_ENABLE_CUDA=ON -
    DUMPIRE_ENABLE_OPENMP=ON -DUMPIRE_ENABLE_MPI=ON -
    DUMPIRE_ENABLE_IPC_SHARED_MEMORY=ON -DUMPIRE_ENABLE_MPI3_SHARED_MEMORY=ON -
    DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES="86;120"  -
    DUMPIRE_ENABLE_C=ON -DCMAKE_INSTALL_PREFIX=/opt/umpire ../

make -j$(nproc)
sudo make install
```

LD_LIBRARY_PATH and PATH changes in ~/.bashrc:

```
export PATH=/opt/paraview/bin:/opt/umpire/bin:/opt/ompi/bin:/opt/prrte/bin:/opt
    /pmix/bin:/opt/ucx/bin:$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=/opt/paraview/lib:/opt/umpire/lib:/opt/ompi/lib:/opt/
    prrte/lib:/opt/pmix/lib:/opt/ucx/lib:/opt/ucx/lib/ucx:$CUDA_HOME/lib64
```

Umpire etc/prefs.sh Changes

```
export UMPIRE_ARCH_PATH=/opt/umpire
```

Hypre

```
cd repos
git clone --recursive https://github.com/hypre-space/hypre.git
cd hypre
mkdir build && cd build
cmake -DBUILD_SHARED_LIBS=ON -DCMAKE_POSITION_INDEPENDENT_CODE=ON -
    DCMAKE_BUILD_TYPE=Release -DCMAKE_C_COMPILER=mpicc -DCMAKE_CXX_COMPILER=
    mpicxx -DHYPRE_ENABLE_CUDA=ON -DMPI_INCLUDE_PATH=/opt/ompi/include -
    DCMAKE_CUDA_ARCHITECTURES="86;120"  -DHYPRE_ENABLE_GPU_AWARE_MPI=ON -
    DHYPRE_ENABLE_OPENMP=ON -DHYPRE_ENABLE_UMPIRE=ON -DHYPRE_ENABLE_UMPIRE_HOST
    =ON -DHYPRE_ENABLE_UMPIRE_DEVICE=ON -DCMAKE_INSTALL_PREFIX=/opt/hypre -
    DTPL_UMPIRE_INCLUDE_DIRS=/opt/umpire/include -DTPL_UMPIRE_LIBRARIES=/opt/
    umpire/lib/libumpire.so ../src
make -j$(nproc)
sudo make install
```

Hypre etc/prefs.sh Changes

```
export HYPRE_ARCH_PATH=/opt/hypre
```

CGAL

```
sudo apt update
sudo apt install libcgal-dev libgmp-dev libmpfr-dev libboost-system-dev
```

CGAL `prefs.sh` Changes

```
#can use default cgal
export CGAL_ARCH_PATH=/usr
```

FFTW

```
# Use lscpu to identify supported architectures to enable
lscpu

cd fftw-3.3.11
mkdir build && cd build
../configure --prefix=/opt/fftw --enable-openmp --enable-shared --enable-
    threads --enable-mpi --enable-sse2 --enable-avx --enable-avx2 --enable-
    avx512
make -j$(nproc)
sudo make install
```

KaHIP

```
mkdir build && cd build

cmake -DCMAKE_INSTALL_PREFIX=/opt/kahip -DCMAKE_BUILD_TYPE=Release -
    DBUILD_SHARED_LIBS=ON -DMPI_HOME=/opt/ompi -DCMAKE_INSTALL_LIBDIR=/opt/
    kahip/lib ..

make -j$(nproc)
sudo make install
```

Karypis Labs

GKlib

```
cd GKlib
mkdir build && cd build

cmake -DCMAKE_INSTALL_PREFIX=/opt/karypis -DCMAKE_BUILD_TYPE=Release -DSHARED=
    ON -DOPENMP=ON ..

make -j$(nproc)
sudo make install
```

METIS

METIS has a couple of bugs in root folder `CMakeLists.txt`:

```
(line 55) include_directories(include)
...
(line 73) add_subdirectory("include")
```

Cmake

```
cd METIS
mkdir build && cd build

cmake -DCMAKE_BUILD_TYPE=Release -DSHARED=ON -DOPENMP=ON -DCMAKE_INSTALL_PREFIX
    =/opt/karypis -DGKLIB_PATH=/opt/karypis ..

make -j$(nproc)
sudo make install
```

Post-Installation Step

Make sure to uncomment the `#defines` for `IDXTYPEWIDTH` and `REALTYPEWIDTH` inside `/opt/karypis/include/metis.h`:

```
(line 33) #define IDXTYPEWIDTH 32
...
(line 43) #define REALTYPEWIDTH 64 /* note 64 for double precision */
```

ParMETIS

```
cmake -DCMAKE_BUILD_TYPE=Release -DSHARED=ON -DOPENMP=ON -DCMAKE_INSTALL_PREFIX
    =/opt/karypis -DGKLIB_PATH=/opt/karypis -DMETIS_PATH=/opt/karypis -
    DCMAKE_C_COMPILER=mpicc ..
```

Scotch

```
# Need Scotch for orthogonal decomposition
git clone --recursive https://gitlab.inria.fr/scotch/scotch.git
```

Cmake

```
cd Scotch
mkdir build && cd build

cmake -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=ON -DCMAKE_INSTALL_PREFIX
    =/opt/scotch -DINSTALL_METIS_HEADERS=OFF  -DMPI_THREAD_MULTIPLE=ON -
    DCMAKE_C_COMPILER=mpicc ..

make -j$(nproc)
sudo make install
```

Scotch `etc/prefs.sh` Changes

```
export SCOTCH_ARCH_PATH=/opt/scotch
```

Zoltan

```
cd Zoltan
mkdir build && cd build

# Use CFLAGS and CXXFLAGS to force shared libraries
../configure --prefix=/opt/zoltan --enable-mpi --with-mpi=/opt/ompi --with-
    parmetis=1 --with-parmetis-incdir=/opt/karypis/include --with-parmetis-
    libdir=/opt/karypis/lib --with-scotch=1 --with-scotch-incdir=/opt/scotch/
    include --with-scotch-libdir=/opt/scotch/lib CFLAGS="-O3 -fPIC" CXXFLAGS="-
    O3 -fPIC"

make everything -j$(nproc)

# Create the shared library version of libzoltan manually
cd src
mpicxx -shared -o libzoltan.so all_allo.o coloring.o color_test.o bucket.o
    g2l_hash.o graph.o divide_machine.o get_processor_name.o ha_ovis.o hier.o
    hier_free_struct.o hsfc_box_assign.o hsfc.o hsfc_hilbert.o
    hsfc_point_assign.o lb_balance.o lb_box_assign.o lb_copy.o lb_eval.o
    lb_free.o lb_init.o lb_invert.o lb_migrate.o lb_part2proc.o lb_point_assign
    .o lb_remap.o lb_set_fn.o lb_set_method.o lb_set_part_sizes.o matrix_build.
    o matrix_distribute.o matrix_operations.o matrix_sym.o matrix_utils.o order
    .o order_struct.o order_tools.o hsfcOrder.o perm.o par_average.o par_bisect
    .o par_median.o par_median_randomized.o par_stats.o par_sync.o
    par_tflops_special.o assign_param_vals.o bind_param.o check_param.o
    free_params.o key_params.o print_params.o set_param.o build_graph.o
    postprocessing.o preprocessing.o scatter_graph.o third_library.o
    verify_graph.o parmetis_interface.o phg_build.o phg_build_calls.o phg.o
    phg_lookup.o phg_verbose.o phg_coarse.o phg_comm.o phg_distrib.o phg_gather
    .o phg_hypergraph.o phg_match.o phg_order.o phg_parkway.o phg_patoh.o
    phg_plot.o phg_rdivide.o phg_refinement.o phg_scale.o phg_serialpartition.o
     phg_util.o phg_tree.o phg_Vcycle.o box_assign.o create_proc_list.o
    inertial1d.o inertial2d.o inertial3d.o point_assign.o rcb_box.o rcb.o
    rcb_util.o rib.o rib_util.o shared.o reftree_build.o reftree_coarse_path.o
    reftree_hash.o reftree_part.o scotch_interface.o block.o cyclic.o random.o
    timer_params.o comm_exchange_sizes.o comm_invert_map.o comm_do.o
    comm_do_reverse.o comm_info.o comm_create.o comm_resize.o comm_sort_ints.o
    comm_destroy.o comm_invert_plan.o zoltan_timer.o timer.o DD_Memory.o
    DD_Find.o DD_Destroy.o DD_Set_Neighbor_Hash_Fn3.o DD_Remove.o DD_Create.o
    DD_Update.o DD_Stats.o DD_Hash2.o DD_Print.o DD_Set_Neighbor_Hash_Fn2.o
    DD_Set_Hash_Fn.o DD_Set_Neighbor_Hash_Fn1.o mem.o zoltan_align.o zoltan_id.
    o zz_coord.o zz_gen_files.o zz_hash.o murmur3.o zz_map.o zz_heap.o zz_init.
    o zz_obj_list.o zz_rand.o zz_set_fn.o zz_sort.o zz_struct.o zz_back_trace.o
     zz_util.o -L/opt/scotch/lib -lptscotch -lscotch -L/opt/karypis/lib -
    lparmetis -lmetis

# should have probably just used
mpicxx -shared -o libzoltan.so *.o -L/opt/scotch/lib -lptscotch -lscotch -L/opt
    /karypis/lib -lparmetis -lmetis

# Create the system installation folders
```

```
sudo mkdir -p /opt/zoltan/lib /opt/zoltan/include

# Copy shared object library
sudo cp libzoltan.so /opt/zoltan/lib/

# Copy the Zoltan_config.h file into include
sudo cp include/Zoltan_config.h /opt/zoltan/include

# Copy the Zoltan header files into include
cd ../../src
sudo cp include/*.h /opt/zoltan/include
```

AMGX

Test launcher Issues

Ended up changing line 55 in `src/CMakeLists.txt` prior to running `cmake`:

```
target_link_libraries(amgx_tests_launcher "/opt/ompi/lib/libmpi.so")
```

Build

```
git clone https://github.com/amgx/amgx.git
cd amgx
mkdir build && cd build

cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=/opt/amgx -
    DCMAKE_CUDA_ARCHITECTURES="86;120" -DCMAKE_C_COMPILER=mpicc -
    DCMAKE_CXX_COMPILER=mpicxx -DCMAKE_CUDA_FLAGS="-I/opt/ompi/include" -
    DCMAKE_EXE_LINKER_FLAGS="-L/opt/ompi/lib -lmpi" ..

make -j$(nproc)
sudo make install
```

PETSc

Repository and Prerequisites

```
# Need PETSc for Scientific Computation
git clone --recursive https://github.com/petsc/PETSc.git
sudo apt install libopenblas-dev liblapack-dev
```

Configure

```
cd PETSc
mkdir build

sudo mkdir /opt/petsc
sudo chown skooby:skooby /opt/petsc

# works after removing Scotch's duplicate metis.h insertion
```

```
./configure --prefix=/opt/petsc --with-openmp=1 --with-bison=1 --with-boost=1
    --with-cuda=1 --with-ucx-dir=/opt/ucx --with-mpi-dir=/opt/ompi --with-amgx-
    dir=/opt/amgx --with-hwloc-dir=/opt/hwloc --with-umpire-dir=/opt/umpire --
    with-scotch-dir=/opt/scotch --with-ptscotch-dir=/opt/scotch --with-fftw-dir
    =/opt/fftw --with-zoltan-dir=/opt/zoltan --with-hypre-dir=/opt/hypre --with
    -parmetis-dir=/opt/karypis --with-metis-dir=/opt/karypis

make PETSC_DIR=/media/skooby/data/repos/petsc PETSC_ARCH=arch-linux-c-debug all

make PETSC_DIR=/media/skooby/data/repos/petsc PETSC_ARCH=arch-linux-c-debug
    install

sudo chown root:root /opt/petsc
```

PETSc `etc/prefs.sh` Changes

```
export PETSC_ARCH_PATH=/opt/petsc
```

OpenFOAM

Haswell Issues

Currently on Haswell, there are issues with the external tools and libraries because I set `WM_THIRD_PARTY_DIR` to the empty string during the OpenFOAM configuration, build, and install (Allwmake). I did this because there are no external tools for OpenFOAM to configure on either machine (in theory).

Note that PETSc's wrapper library, `libpetscFoam.so`, has been built and installed in an arbitrary location, outside of path conventions. Several other tools may not be functional for perhaps similar reasons. OpenFOAM's `lib` folder contains a `dummy` folder with a number of similar wrapper libraries to `libpetscFoam.so`, but for METIS, Scotch, Zoltan, etc. Thus, these wrapper libs may not be functional either.

Copy `prefs.sh` into `OpenFOAM/etc/`

See the OpenFOAM External Tools Introduction for details and possible issues.

Edit the .bashrc

```
source /media/skooby/data/repos/OpenFOAM_com/OpenFOAM/etc/bashrc
export WM_THIRD_PARTY_DIR=/opt
```

Run Allwmake

```
cd OpenFOAM

# Executes both Allwmake and Allwmake-modules
./Allwmake -j

# ran 7/25 for just the main script
# ./Allwmake -prefix=false

# Plugins
/./Allwmake-plugins -j
```

AmgX4Foam

Hacks

AmgX4Foam's `src/csrMatrix/csrMatrix.h` was missing a `#include <cuda_runtime.h>` statement.

Out of the box, AmgX4Foam's default configuration options limit only a single CUDA arch code, so I changed the behavior of the `-cu` flag so that it is no longer responsible for passing the architecture. Instead, I replaced line 12 in the `wmake/cuda` file: `cuARCH := -m64 -arch=sm_$(NVARCH)` with `cuARCH := -m64 -gencode arch=compute_86,code=sm_86 -gencode arch=compute_120,code=sm_120`. I still had to enable CUDA by including the flag though: `./Allwmake -cu 1`.

Lines 17 in `src/Make/options` and 12 in `wmake/c++` were also removed to complete the build.

I also had to add 3 lines to the top of `src/Make/options`:

```
LIB_SRC = $(FOAM_SRC)
AMGX_INC = /opt/amgx/include
AMGX_LIB = /opt/amgx/lib
```

Finally, I had to add these two lines to `EXE_INC` and `LIB_LIBS` accordingly:

```
EXE_INC = \
    ...
    -I/usr/local/cuda/include \
    -I/opt/ompi/include \
    ...

LIB_LIBS = \
    ...
    -L/usr/local/cuda/lib64 -lcudart \
    ...
```

By default, this installs to the `FOAM_USER_LIBBIN` folder, whether or not that folder exists. `FOAM_USER_LIBBIN` is one of the non-standard directories using OpenFOAM's variable-based naming convention: `~/OpenFOAM/<user name>-<version>/platforms/<arch><compiler><precision><index size><third party folder>/lib`.

```
./Allwmake
```

Final `~/.bashrc` Settings

```
export CUDA_HOME=/usr/local/cuda
export PATH=/opt/scotch/bin:/opt/karypis/bin:/opt/fftw/bin:/opt/umpire/bin:/opt
    /paraview/bin:/opt/ompi/bin:/opt/prrte/bin:/opt/pmix/bin:/opt/hwloc/bin:/
    opt/libevent/bin:/opt/ucx/bin:$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=/opt/petsc/lib:/opt/zoltan/lib:/opt/scotch/lib:/opt/
    karypis/lib:/opt/amgx/lib:/opt/kahip/lib:/opt/fftw/lib:/opt/hypre/lib:/opt/
    umpire/lib:/opt/paraview/lib:/opt/ompi/lib:/opt/prrte/lib:/opt/pmix/lib:/
    opt/hwloc/lib:/opt/libevent/lib:/opt/ucx/lib:$CUDA_HOME/lib64

source /media/skooby/data/repos/OpenFOAM_com/OpenFOAM/etc/bashrc
```